

ASTRA - Diagnostic indépendant et définition du facteur cinq

Conclusion : le facteur cinq reste un objectif expérimental, pas une propriété démontrée du générateur actuel. Le meilleur investissement consiste à mesurer le chemin critique, puis à spécialiser les formes réellement exécutées et réduire les attentes. Réécrire tout PyTorch autour des poids serait prématuré. Cette analyse est exclusivement documentaire et statique ; aucun benchmark, serveur, cache ou code produit n'a été modifié. Les références fichiers:lignes correspondent aux sources consultées le 7 octobre 2026.

La production charge trois réseaux PyTorch CUDA BF16, sans compilation, dans `terrain_app.py:59` et `terrain_server.py:175`. Le parcours va du mouvement navigateur aux abonnements et requêtes, puis au worker partagé, au bootstrap CPU natif, au conditionnement, au coarse appris, aux deux passes latentes, au decoder, au transport physique et au rendu WebGPU. `webgpu/` est un probe d'inférence séparé : optimiser ONNX/WASM ne traiterait pas directement ce parcours. Le bootstrap terrestre actuel est un raster signé 1024 × 512 ; les anciens Macro A4 sont retirés, comme l'indique `README.md`.

Faits mesurés antérieurement : `docs/RUNTIME_LOD_OPTIMIZATION.md` rapporte, sur natural, un LOD 3 neuf médian de 2,117 s après optimisation, contre 2,978 s avant ; LOD 2 reste variable, avec un run final à 5,356 s. Le gain apparié de 15,1 % au LOD 2 est exploratoire, notamment faute de snapshot exact du harness intermédiaire. Les 92 arrays égaux octet par octet établissent une fidélité limitée au corpus et à l'environnement testés. Ils n'établissent aucun nouveau débit terrestre.

Dans `E:/TerrainDiffusionRuntime/terrestrial-bootstrap-runtime/`, `browser-final-cold.json` conserve 12,443 s de complétion après zoom, 22 misses physiques ; `browser-final-warm.json` conserve 0,570 s et 22 hits. Ce ratio exprime une revisite, pas une accélération de génération. Leurs chronomètres d'aperçu et de zoom ont des origines distinctes. La vérification actuelle des hashes du reçu froid trouve des différences pour `index.html`, `terrain_renderer.js`, `terrain_inference.py` et `terrain_server.py` : ce sont des preuves historiques, pas un profil exact des sources présentes. `cpu-bootstrap.json` rapporte earthlike à 1,182 s de première création, dont 0,596 s natif, et 0,037 s de relecture vérifiée, sur une seule seed.

La métrique principale doit être le temps mouvement→couverture native visible d'un viewport fixé, en région jamais calculée, modèles résidents. Mesurer séparément démarrage complet, première amélioration, débit de surface nouvelle et revisites. Le temps NN comprend les trois modèles ; le temps global comprend aussi files, dépendances, CPU, transferts et présentation.

Déduction d'Amdahl : $S = 1 / ((1-p) + p/s)$. Pour x5 global, p doit dépasser 80 % ; si p=90 %, le sous-système doit atteindre x9 ; si p=95 %, x6,33. Accélérer seulement le NN x5 avec p=90 % donne x3,57 global. Les objectifs illustratifs seraient 0,423 s contre 2,117 s, ou 2,489 s contre 12,443 s, uniquement après reconstruction de baselines comparables. Additionner des durées de stages imbriquées ou multiplier des gains corrélés produirait un budget fictif.

Moteur spécialisé : ce que les poids permettent réellement d'amortir

Un checkpoint fixe permet de préparer les constantes et d'optimiser des formes connues ; il ne permet pas de précalculer les activations dépendant du bruit, du climat et de la région. `terrain_inference.py:88` cache déjà normalisation et gain des poids MP, avec leurs arrondis historiques. Les embeddings coarse sont réutilisés à `terrain_inference.py:165`. `terrain_nn_constants.py:40`, `:52` et `:71` mémorise coefficients MP et filtres. Ces économies sont déjà dans la baseline ; un moteur dédié ne peut pas les revendiquer une deuxième fois.

La spécialisation la plus directe concerne les formes : coarse $11 \times 64 \times 64$, base $5 \times 64 \times 64$, decoder $5 \times 512 \times 512$ dans `benchmark_inference.py`. Coarse et decoder restent batch 1 ; le base utilise généralement batch 16. `terrain_inference.py:405` interdit explicitement les batches coarse/decoder supérieurs à un hors expérimentation : ils ont modifié les mètres du terrain. Les 19 forwards base du LOD 3 historique comprennent 17 batches 16 et deux batches 9. Pourtant les CUDA graphs ne couvrent le base que jusqu'à quatre, et le decoder reste eager (`terrain_inference.py:418`). Capturer 16, 9 et le decoder constitue donc une expérience prioritaire, sans changement volontaire des partitions.

`terrain_cuda_graphs.py:47` conserve entrées statiques, copies, verrou, événement et clone de sortie ; chaque bucket effectue trois warmups et une comparaison exacte. Les gains possibles concernent les lancements et trous de soumission, pas les FLOPs intrinsèques des convolutions. Tester aussi un graphe incluant les vingt étapes coarse, actuellement exécutées dans une boucle Python, réclamerait un état de solveur statique et une protection contre la réutilisation de buffers encore vivants. Capturer davantage augmente mémoire et premier usage ; une victoire à chaud peut dégrader l'ouverture.

PyTorch compilé est une alternative expérimentale plus limitée qu'une réécriture : compiler les forwards et préparations compatibles, conserver orchestration et checkpoint. Le profil local rejette actuellement `torch_compile` (`terrain_inference.py:414`), et `world_pipeline.py:336` le désactive sous Windows. Ce sont des restrictions du code inspecté, pas une affirmation générale sur toutes les versions disponibles. TensorRT représente une autre expérience de conversion, à évaluer dans un environnement isolé ; couverture d'opérateurs, précision effective, coût de construction et reproductibilité demeurent inconnus ici. Aucun moteur de ce type n'a été mesuré.

Les kernels ciblés sont plus justifiables lorsqu'un profil attribue le coût aux allocations et opérations élémentaires : normalisation, activation, somme MP, concaténation pondérée, modulation conditionnelle. Attention : remplacer multiplication BF16 puis réduction par une accumulation fusionnée peut changer l'arrondi. `terrain_nn_constants.py:84` matérialise l'attention du seul midblock base (8×8 au runtime) par deux einsum et softmax ; l'alternative SDPA dans `UNET_block.py:110` se trouve après un return et n'est donc pas active. Une conversion doit vérifier axes, échelle et résultats, sans promesse automatique de gain.

Hypothèse bornée : éliminer totalement un poste représentant 10 %, 20 % ou 40 % du temps donne au maximum $\times 1,11$, $\times 1,25$ ou $\times 1,67$. Une spécialisation complète devient rationnelle seulement si plusieurs postes dominants restent coûteux après graphs et compilation, avec un gain incrémental mesuré. Les convolutions génériques demeurent d'abord la référence ; aucune sparsité exploitable des poids n'est établie.

Génération, mathématiques, files et rendu : réduire le travail utilement

Le bootstrap natif essaie jusqu'à douze candidats déterministes (native/terrain_bootstrap/src/main.rs:137). Le cœur voisin utilise déjà Rayon (world-core/src/lib.rs:922). Le wrapper exporte les pixels en boucle ; géométrie et trigonométrie fixes pourraient être précalculées. Préserver le premier candidat acceptable, la reconstruction aux frontières du cube et les mêmes réductions est obligatoire. terrain_bootstrap.py:161 trie les distributions ETOPO ; :197 calibre séparément terre et profondeur ; :302 compresse et persiste. Un cache persistant exact des quantiles source améliorerait surtout le démarrage, sans éliminer les tris propres au nouveau monde.

terrain_conditioning.py:180 interpole les quantiles climatiques par latitude ; :196 évalue quatre bruits et applique le gradient thermique. Préparer les grilles et tables invariantes, puis mettre en cache des blocs alignés, est plausible. Conserver coordonnées, unités, racine signée et ordre d'arrondi. La transformation finale $h = \text{sign}(z)z^2$ amplifie une erreur δz approximativement de $2|z|\delta z$: une faible erreur latente ne garantit pas un écart inférieur au mètre. Le bruit portable PCG/Marsaglia, déjà compilé par Numba dans portable_rng.py:58, ne peut être remplacé librement par un RNG CUDA.

La géométrie amène un recouvrement structurel : fenêtres latentes 64/32, coarse 64/48, decoder 512/384. Les rapports surfaciques intérieurs valent respectivement quatre et environ 1,78 ; les deux passes latentes ajoutent leurs dépendances. Ces rapports ne sont pas des gains récupérables : chevauchement, padding et fusion assurent le résultat appris. world_pipeline.py:1231 construit les deux passes ; :1375 reconstruit le DEM par débruitage laplacien. terrain_window_scheduler.py:77 observe les dépendances sans les réordonner. Regrouper arbitrairement plusieurs tuiles changerait les batches BF16 et possiblement le monde ; conserver leurs partitions constitue la première contrainte.

Une piste concrète est la persistance coarse synchrone : terrain_coarse.py:291 synchronise CUDA, puis :404 copie vers CPU, écrit NPY et sidecar, et vérifie le résultat dans le chemin de calcul. Une file bornée de copies et écritures, avec publication atomique après validation, pourrait recouvrir ces attentes. Même logique pour terrain_server.py:804 puis :832 : une tuile neuve est écrite avant d'être relue pour HTTP. Transmettre directement le résultat immuable supprimerait cette relecture ; la durabilité resterait explicitement suivie.

terrain_jobs.py:155 dispose d'une seule voie compute et de deux encodeurs ; le sémaphore peut attendre avant compute. index.html:103 limite à quatre transferts et :80 cadence les admissions à 50 ms. Les barrières LOD $4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$ exigent un dessin soumis avant l'étape suivante. Ces mécanismes protègent la progression mais peuvent créer des attentes ; augmenter seulement HTTP ne parallélise pas le GPU. Le climat utilise une grille compacte 33^2 , avec contexte coarse et régression locale (terrain_climate.py:15), susceptible d'être réutilisée entre voisins.

Chaque réponse contient 391 444 octets physiques ; terrain_renderer.js:180 transfère hauteur/climat, exécute deux filtres puis l'ombrage une fois. Les textures coûtent 653 588 octets par tuile, hors scratch et copies hôte. Optimiser uploads, réutilisation et évictions compte surtout pour revisites/grands écrans. Les caches GPU, limités à 512 Mio par monde dans terrain_server.py:207, partagent la VRAM avec modèles, graphs et navigateur : augmenter leur budget peut réduire régénérations ou empirer la contention. Aucun facteur global n'est attribuable sans profil.

Plan expérimental, décisions x5 et coût de conservation du monde

Priorité zéro : construire une baseline actuelle lors d'une campagne dédiée, serveur arrêté pour les microbenchmarks et un seul workload NN. Archiver sources importées, versions, driver, GPU, hashes des poids, identités natives et paramètres ; conserver modifications locales et harness exact. Séparer modèle froid, modèles résidents/stores neufs, coarse persistant, cache physique et cache navigateur. Inclure natural protégé et les quatre styles terrestres, sans mélanger leurs distributions temporelles.

Instrumenter une chronologie commune : événement caméra, publication des intérêts, admission, attente compute, attente encodeur, conditionnement CPU, RNG, copies H2D/D2H, kernels, fusion, persistance, réponse, upload et premier dessin couvrant. Les événements CUDA ne prouvent pas que chaque intervalle est du calcul pur ; un dessin soumis ne prouve pas la présentation écran.

benchmark_runtime_lod.py et benchmark_runtime_progression.py constituent des bases distinctes, tandis que benchmark_runtime_navigation.cjs observe le produit. benchmark_coarse_world.py demande encore LOD 12 : le protocole doit être actualisé avant réutilisation. Mesurer d'abord avec instrumentation, puis confirmer sans son surcoût.

Priorité un : expériences isolées avec retour arrière. Capturer les batches base 16/9 et decoder ; mesurer capture, VRAM et débit stabilisé. Sortir la persistance coarse du chemin critique avec backpressure. Éviter le cycle écriture/relecture HTTP pour les misses. Préparer le bruit portable du prochain batch, en gardant partition et flux identiques. Publier temps retiré en secondes et fraction du total, y compris résultats nuls ou défavorables. Budget indicatif : quelques jours chacun, sans engagement de résultat.

Priorité deux : comparer compilation ciblée, conversion TensorRT et kernels spécialisés sur les mêmes entrées. Estimation : une à deux semaines pour évaluer un backend ; plusieurs semaines pour une chaîne dédiée maintenable. Ajouter compilation, conversion, warmup, stockage des artefacts, invalidation et compatibilité matérielle au coût total. Seuil de rentabilité : nombre d'appels supérieur au coût initial supplémentaire divisé par l'économie par appel. Pour une session courte, un meilleur débit peut être un mauvais produit.

Les gates qualité doivent porter sur DEM et cinq canaux physiques, pas seulement tensors internes : égalité octet par octet visée ; à défaut, écarts maximum/moyen/quantiles, seuil hauteur historique de 1 m et limites climatiques documentées. Toute tolérance exige une identité distincte et une validation explicite. Vérifier côtes, bathymétrie, reliefs, températures polaires, raccords, coordonnées négatives, seeds u64, annulation/reprise, requêtes inversées et pression mémoire. Les partitions BF16 actuelles ne démontrent pas une indépendance universelle à l'ordre : en faire un test bloquant, sans la présumer.

Décision x5 : exiger, sur parcours appariés répétés et ordre des versions alterné, une réduction d'au moins 80 % du temps global médian, avec dispersion publiée et absence de régression perceptible des premiers aperçus. Un petit corpus ne certifie pas le p95 ; prolonger ensuite en exploration continue. Si le plancher non optimisé dépasse 20 %, abandonner la promesse x5 par seul backend. Quantification INT8, moins d'étapes, autre recouvrement ou distillation sont des branches approximatives distinctes, jamais la solution implicite. Livrer alors le meilleur gain exact démontré et le coût restant, plutôt qu'un facteur théorique composé.